

Numerical Mathematics II for Engineers

Tutorial 13

Topics : High-order schemes, reconstruction and limiters, monotone schemes

Discussion in the tutorials of the week 2 – 6 February 2026

Disclaimers:

- To test your code, we provide tests. This should help you to locate precisely your errors, and code more efficiently. To run the test, you need to install pytest.
- Exercises should be solved in **fixed groups of 3 students**.
- Hand in the solution in **one folder** labeled `hw[hw_number]_group[group_number]` and containing:
 - **One pdf** for the theoretical questions and comments on the numerical results,
 - **One python file per programming exercise**.
 - Write the group number and all names of your members **in each file**.

Exercise 13.1: Programming: high order schemes with slope limiters

The goal of this problem is to implement a second-order high-resolution scheme for solving the linear advection

$$u_t + au_x = 0,$$

with a constant.

We use the method of line described in the lecture with the upwind flux. To achieve the second-order convergence, we need two main ingredients: a reconstruction in space, and a time-stepping method to evolve the piecewise linear approximated solution.

- *Reconstruction in space:* given the averaged values on the cells, we reconstruct a piecewise linear approximation of the solution. We want to compare the behavior for the following slope reconstructions:
 - constant in space (i.e. slopes = 0),
 - unlimited central difference quotient,
 - minmod limiter,
 - MC limiter.

- *Time stepping*: for the time stepping we use the second-order Heun method

$$\begin{aligned}U^{(1)} &= U^n + \Delta t \mathcal{L}(U^n), \\U^{(2)} &= U^{(1)} + \Delta t \mathcal{L}(U^{(1)}), \\U^{n+1} &= \frac{1}{2} (U^n + U^{(2)}).\end{aligned}$$

We provide the template `high_order_template.py`. For comparison, the first order method with upwind flux and forward Euler time stepping is already implemented.

Implementation:

- Implement the time stepping scheme `advance_in_time_heun`.
- Implement the functions `minmod2` and `minmod3` that compute the minmod of 2, respectively 3, arguments.
- Implement the function `slope_limiters`, calling if necessary the minmod functions implemented before.
- In the time stepping loop, implement the right hand side `fu_rhs()`.

Hints for the implementation:

- Go through the implementation of the first-order method and try to follow the same logic for the second-order method.
- Instead of passing U_i^n and U_{i+1}^n to compute the numerical flux, you need to pass the values of the reconstructions on cells i and $i+1$ at $x_{i+1/2}^-$ and $x_{i+1/2}^+$, respectively.
- In each real time step there are two stages for our Heun method, and we need to reconstruct slopes. For the first-order method, we needed 1 ghost cell at each domain end. Now, in each stage, we need one ghost cell on each side for slope reconstruction and another one for flux computation, i.e., 2 on each side of the domain. Don't forget to update the ghost values at the beginning of each stage!
- Use the tests provided in `high_order_test.py` to test each individual functions.

We now consider 2 different test problems on the domain $[0, 1]$:

- A discontinuous test function

$$u_0(x) = \begin{cases} 1, & \text{if } 0.25 < x < 0.75, \\ 0, & \text{otherwise.} \end{cases}$$

- A smooth test function

$$u_0(x) = \sin(2\pi x).$$

For both test problems (the smooth and the discontinuous) and your 4 different methods of slope reconstruction (use CFL=0.4 and $a = 1$):

- plot both your computed and true solution at $T = 1$ for $N = 80$. What do you observe? How diffusive are the methods around the peak for the smooth function? Is there overshoot for the discontinuous test function?

- (f) compute and report convergence rates in the L^1 and L^∞ norm for $N=20,40,80,160,320,640,1280,2560$. Do the rates meet your expectation? Why or why not?

Which method does best overall?

Exercise 13.2: Linear reconstruction

We consider the approximate solution computed using a finite volume scheme and shown in Figure 1.

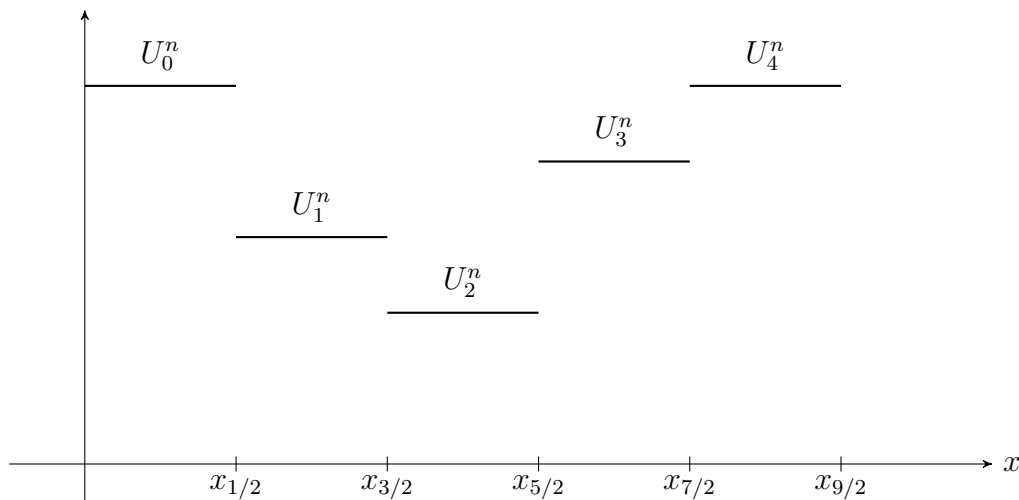


Figure 1: Approximated solution U^n .

- Draw the reconstructed solution using the minmod limiter.
- Draw the reconstructed solution using the MC limiter.

Exercise 13.3: Monotone schemes

In the lecture you have learned about monotone fluxes, and the definition will be extended to schemes. We recall the notation for a numerical schemes with a 3-stencil approximating $u_t + f(u)_x = 0$:

$$U_i^{n+1} = \phi(U_{i-1}^n, U_i^n, U_{i+1}^n).$$

We say a *scheme* $(U, V, W) \mapsto \phi(U, V, W)$ is *monotone* when it is a non-decreasing function in each of its arguments:

$$\frac{\partial \phi}{\partial U} \geq 0, \quad \frac{\partial \phi}{\partial V} \geq 0, \quad \frac{\partial \phi}{\partial W} \geq 0.$$

Show that if a scheme of the form

$$U_i^{n+1} = U_i^n - \frac{\Delta t}{h} [\mathcal{F}(U_i^n, U_{i+1}^n) - \mathcal{F}(U_{i-1}^n, U_i^n)]$$

for solving is monotone, then the flux $\mathcal{F}$ of this scheme is monotone.